EC413 Lab 7: Pipelined CPUs

Adding Data Forwarding to a Five-Stage MIPS Pipeline

Jatin Punjabi

November 13, 2026

1 Overview

The supplied design is a standard five-stage MIPS pipeline (IF, ID, EX, MEM, WB) with no forwarding, so an instruction that reads a register written by one of the previous two instructions reads a stale value. This lab adds "1 ahead" and "2 ahead" forwarding, arbitration between them, protection for writes to `$0` and for instructions that do not write a register, a register-file bypass, and (extra credit) forwarding of store data for a `lw` followed by `sw`.

All changes were simulated with Icarus Verilog using the lab's `discussion_tb.v` and a self-checking testbench, `fwd_tb.v`. Source files are in `src/`; unchanged supplied files are in `provided/`.

2 Task 1: Pre-lab (baseline simulation)

The unmodified project was run with the 10-instruction sequence in `discussion_tb.v`. Without forwarding, every dependent instruction reads a stale register, giving exactly the "wrong" values noted in the testbench comments (Table 1).

Instruction	Result	Expected	Baseline	With forwarding
`add $R5,$R1,$R4`	R5	569	423	569
`slt $R6,$R3,$R5`	R6	1	0	1
`sub $R7,$R4,$R6`	R7	145	146	145
`sw $R7,0($R0)`	Mem[0]	145	0	145
`add $R8,$R7,$R2`	R8	237	92	237

Table 1: Final values before and after the changes.

Note. Two port declarations in the supplied code had to be made consistent for a strict simulator to compile it: `BranchMux` in `pipecontrol.v` (port `[1:0]`, reg `[2:0]`) and `ex` in `stallmux.v` (port `[6:0]`, reg `[13:0]`). Behaviour is unchanged.

3 Tasks 2–4: Forwarding unit (1 ahead, 2 ahead, arbitration)

A new module `forwardingunit.v` sits in the EX stage. It compares the source registers held in ID/EX (`Rs`, `Rt`) with the destination registers in EX/MEM and MEM/WB and drives `ForwardA` and `ForwardB`. In `CPU.v` two 3-to-1 muxes (`ForwardAMux`, `ForwardBMux`) were inserted between the ID/EX register and the ALU. The existing wires `MuxASelect` and `MuxBSelect` carry the selects.

Select	Source	Meaning
00	ID/EX register value	no hazard
01	`WriteBackData` (after the MemToReg mux)	2 ahead
10	`MemAluOut` (EX/MEM ALU result)	1 ahead

Table 2: Forwarding mux encoding.

Task 2 (1 ahead). If EX/MEM writes a register and `EXMEM_Rd` equals `IDEX_Rs` (or `Rt`), select `10`.

Task 3 (2 ahead). If MEM/WB writes a register and `MEMWB_Rd` equals `IDEX_Rs` (or `Rt`), select `01`. The write-back *mux output* is used, so values that come from a load are also forwarded correctly.

Task 4 (arbitration). When both stages match, the instruction in EX/MEM is the more recent writer, so 1 ahead takes priority; 2 ahead is chosen only if 1 ahead does not match.

Datapath changes. The ALU's A input is driven by `ForwardAMux`; input 0 of the existing AluSrc mux (previously ID/EX `B`) is driven by `ForwardBMux`, so immediates still override `B`. The EX/MEM store-data register is loaded from the forwarded `B` value, so a `sw` also receives forwarded data.

Listing 1: `forwardingunit.v`

```
module forwardingunit(ForwardA, ForwardB,
                      IDEX_Rs, IDEX_Rt,
                      EXMEM_RegWrite, EXMEM_Rd,
                      MEMWB_RegWrite, MEMWB_Rd);

   output [1:0] ForwardA, ForwardB;
   input [4:0] IDEX_Rs, IDEX_Rt, EXMEM_Rd, MEMWB_Rd;
   input EXMEM_RegWrite, MEMWB_RegWrite;

   // Task 2: 1-ahead hazards (producer in MEM stage)
   wire OneAheadA = EXMEM_RegWrite && (EXMEM_Rd != 5'd0) && (EXMEM_Rd == IDEX_Rs);
   wire OneAheadB = EXMEM_RegWrite && (EXMEM_Rd != 5'd0) && (EXMEM_Rd == IDEX_Rt);

   // Task 3: 2-ahead hazards (producer in WB stage)
   wire TwoAheadA = MEMWB_RegWrite && (MEMWB_Rd != 5'd0) && (MEMWB_Rd == IDEX_Rs);
   wire TwoAheadB = MEMWB_RegWrite && (MEMWB_Rd != 5'd0) && (MEMWB_Rd == IDEX_Rt);

   // Task 4: arbitration - 1 ahead has priority over 2 ahead
   assign ForwardA = OneAheadA ? 2'b10 : (TwoAheadA ? 2'b01 : 2'b00);
   assign ForwardB = OneAheadB ? 2'b10 : (TwoAheadB ? 2'b01 : 2'b00);

endmodule
```

4 Task 5: Writes to $0

Register `$0` is hard-wired to zero, but `add $0,$R10,$R10` still has `Rd=0` and `RegWrite=1`, and would be forwarded. Every forwarding condition therefore also requires `Rd != 0`. *Test:* `add $0,$R10,$R10` followed by `add $R11,$0,$R10` yields R11 = 5 (it would be 15 if the `$0` result were forwarded).

5 Task 6: No write

Instructions such as `sw`, `beq`, `bne`, `j` and pipeline bubbles have `RegWrite=0`, but the register field in their encoding can still match a later source register (and a `sw`'s ALU output is only an address). Every forwarding condition therefore requires the producer's RegWrite bit (`EXMEM_WBControl[0]` for 1 ahead, `WB_WBControl[0]` for 2 ahead). *Test:* `sw $R2,8($R0)` followed by `add $R12,$R2,$R2` yields 184 (it would be 16 if the `sw`'s address were forwarded).

6 Task 7: Register-file bypass

A value produced three instructions earlier is being written in WB in the same cycle the consumer reads the register file in ID, so forwarding cannot help; the register file must pass the write data straight through. The supplied `RegFile.v` had write-through, but it ignored `RegWrite` and `RegWrite` was missing from the sensitivity list, so a non-writing instruction could wrongly bypass. It now bypasses only when `RegWrite && WriteRegister == ReadReg && ReadReg != 0`:

```
always @(ReadReg1 or ReadReg2 or WriteRegister or WriteData or RegWrite)
  begin
    ReadData1 = Reg_File[ReadReg1];
    ReadData2 = Reg_File[ReadReg2];
    if (RegWrite && ReadReg1 == WriteRegister && (ReadReg1 != 0))
      ReadData1 = WriteData;
    if (RegWrite && ReadReg2 == WriteRegister && (ReadReg2 != 0))
      ReadData2 = WriteData;
  end
```

Test: `addi $R13,$0,3; nop; nop; add $R14,$R13,$R13` yields R14 = 6.

7 Extra credit: LW followed by SW

A `sw` immediately after the instruction that produces its store data reaches MEM while the producer is in WB, so the data is only available as `WriteBackData`. No stall is needed:

- the `Rt` field is carried one more stage (new register `EXMEM_RtReg`);
- new module `storeforward.v` asserts `ForwardStore` when `MemWrite && RegWrite_WB && WB_Rd != 0 && WB_Rd == EXMEM_Rt`;
- a new mux, `StoreDataMux`, in front of the data-memory write-data port selects `WriteBackData` instead of the EX/MEM store-data register.

Listing 2: `storeforward.v`

```
module storeforward(ForwardStore, MEM_MemWrite, EXMEM_Rt, MEMWB_RegWrite, MEMWB_Rd);
   output ForwardStore;
   input MEM_MemWrite, MEMWB_RegWrite;
   input [4:0] EXMEM_Rt, MEMWB_Rd;

   assign ForwardStore = MEM_MemWrite && MEMWB_RegWrite &&
                         (MEMWB_Rd != 5'd0) && (MEMWB_Rd == EXMEM_Rt);
endmodule
```

Test: `lw $R17,8($R0); sw $R17,20($R0)` stores 92 into Mem[20]. A `lw`, `nop`, `sw` sequence is covered by the ordinary 2-ahead forward on B and also stores the correct value.

Limitation. A `lw` followed directly by a non-store instruction that uses the loaded value (a load-use hazard) still requires a one-cycle stall. `hazardunit.v` is a stub in the supplied project, and this case is outside the lab.

8 Modified datapath

Figure 1 shows the EX and MEM stages with the new hardware in colour: red is the 1-ahead path, green the 2-ahead/write-back paths, and blue the forwarding-unit control.

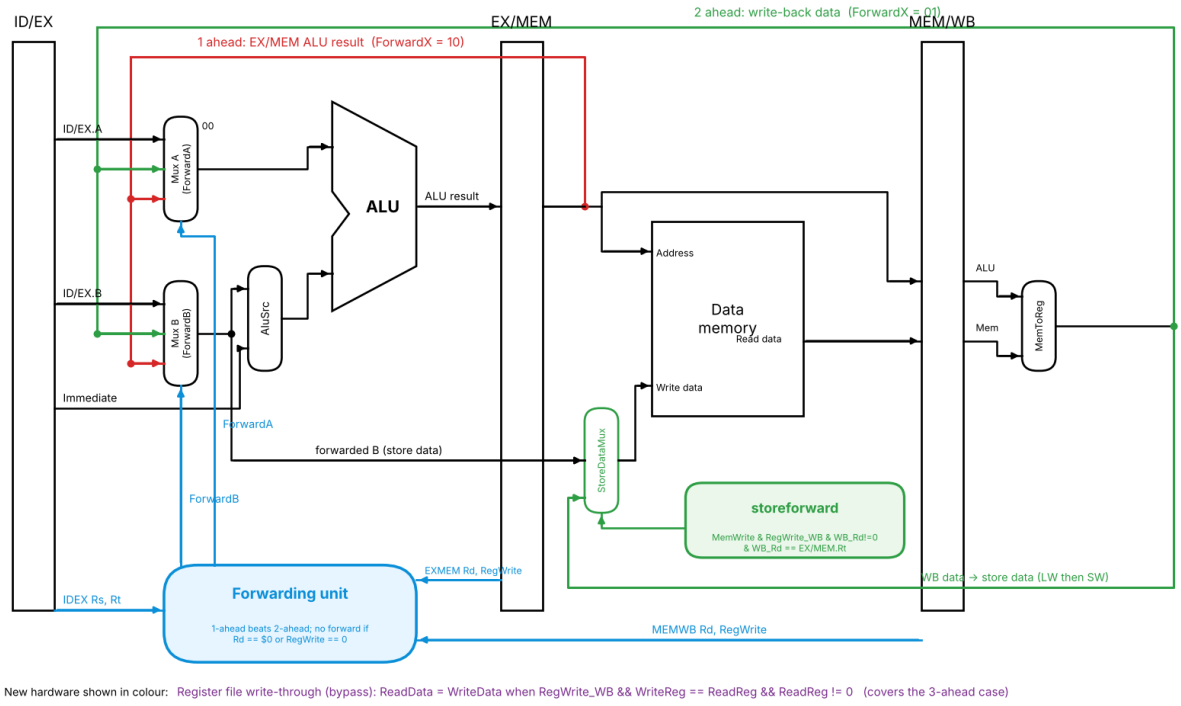


Figure 1: Modified datapath: forwarding muxes and unit, store-data forwarding, and register-file bypass.

9 Verification

`fwd_tb.v` runs the lab's 10 instructions plus the tests described above and checks the register file and data memory. All 14 checks pass. To confirm each feature is actually exercised, the following were disabled one at a time and the testbench was confirmed to fail in each case: the `$0` guard, the RegWrite guard, the arbitration, the 2-ahead path, the register-file bypass, and the LW→SW forward.

Feature	Test	Result
1 ahead	`add`/`slt`/`sub` chain	R5=569, R6=1, R7=145
2 ahead	`add $R8,$R7,$R2`	R8=237
`$0` write	`add $0,...; add $R11,$0,$R10`	R11=5
No write	`sw; add $R12,$R2,$R2`	R12=184
Bypass (3 ahead)	`addi; nop; nop; add`	R14=6
Arbitration	`addi; addi; add` on one register	R16=6
LW→SW	`lw $R17; sw $R17`	Mem[20]=92
LW, nop, SW	`lw $R18; nop; sw $R18`	Mem[21]=92

Table 3: Summary of tests (all pass).

10 Files changed or added

- `CPU.v`: forwarding muxes and unit, store-data mux, EX/MEM Rt register
- `forwardingunit.v`, `storeforward.v`: new modules
- `RegFile.v`: bypass gated by RegWrite
- `pipecontrol.v`, `stallmux.v`: port width declarations made consistent
- `fwd_tb.v`: new self-checking testbench